

Top-Down Parsing via Bidirectional Context Expansion: A Distributional Approach to Constituency

Research Notes, Claude Code prompted by Rob Freeman

January 12, 2026

Abstract

We explore a parsing approach based on *substitution patterns* rather than local co-occurrence. The key insight is that syntactic units should be identified by maximizing the size of their *bidirectional expansion*: the set of units that can substitute in similar contexts, where contexts themselves are also expanded to handle novel sentences. This document traces our reasoning, presents concrete examples, describes our prototype implementation, and identifies current technical challenges.

1 Initial Problem: Local Context vs. Global Substitutability

1.1 The Limitation of Current Approaches

Existing distributional parsers (including our baseline `hdp1.py`) score spans using *local context windows*:

Current approach

```
left_context = tokens[i-4:i]      # 4 tokens to the left
right_context = tokens[i+n:i+n+4] # 4 tokens to the right
span_score = f(left_context, right_context)
```

Problem: This only measures whether a span fits *this particular* 4-token window, not whether it's a true substitutable unit across diverse contexts.

1.2 Example: “my money” vs. “lost my”

Consider the sentence: I lost my money yesterday

Intuition says: “my money” should be a unit (possessive NP)

Current parser logic:

- “lost my”: appears in immediate sequence, has co-occurrence statistics
- “my money”: also appears in sequence, similar statistics
- *Both* have consistent local contexts

The key question: How do we distinguish compositional units from accidental adjacency?

1.3 The Substitution Criterion

Insight: A true syntactic unit should have many *substitutes* across diverse sentence frames.

Frame	Substitutes
I lost _____ yesterday	my money, your keys, his wallet, the car
I need _____ for rent	my money, your help, some cash, the funds
Where is _____ ?	my money, your phone, his address, the key

“My money” has *many diverse substitutes* $\implies$ strong unit

Frame	Substitutes
I _____ money	lost my, found my, need my, want my, ...
You _____ keys	lost your, found your, took your, ...

“Lost my” has *limited substitutes* and they require changing the determiner $\implies$ weaker unit

2 From Single-Sided to Bidirectional Expansion

2.1 First Attempt: Expand Units Only

Naive approach:

1. Fix the context: “did you ____ with Sarah”
2. Find all units from corpus that fit this *exact* context
3. Size of this set = substitutability score

Problem: What if the exact context “did you _ with Sarah” was never seen in the corpus?

2.2 The Critical Insight: Expand Contexts Too

Key realization: Novel sentences will have novel contexts. We can’t match exact contexts; we must match *context patterns*.

2.2.1 Example: Novel Sentence

Sentence: Should we go out on Friday

The exact pattern “should we _ on Friday” might be rare.

Solution: Expand the context pattern:

$$\begin{aligned} \text{Context}_0 &= \text{“should we _ on Friday”} \\ \text{Context expansion} &= \left\{ \begin{array}{l} \text{“should we _ on Friday”} \\ \text{“should we _ tonight”} \\ \text{“did you _ on Wednesday”} \\ \text{“we _ on Wednesdays”} \\ \text{“would you _ with me”} \end{array} \right. \end{aligned}$$

Now find units fitting *any* of these expanded contexts:

$$\text{Unit expansion} = \{ \text{“go out”}, \text{“hang out”}, \text{“meet up”}, \text{“get together”}, \dots \}$$

2.3 Bidirectional Expansion Algorithm

```
def bidirectional_expansion(unit_0, context_0, catalog):
    """
    Simultaneously expand unit and context.
    Returns: (unit_expansion, context_expansion)
    """
    # Initialize
    unit_expansion = {unit_0}
    context_expansion = {context_0}

    for iteration in range(max_iters):
        # EXPAND CONTEXTS:
        # Find contexts similar to current expansion
        new_contexts = set()
        for ctx in context_expansion:
            similar = find_similar_contexts(ctx, catalog)
            new_contexts.update(similar)
        context_expansion.update(new_contexts)

        # EXPAND UNITS:
        # Find units appearing in expanded contexts
        new_units = set()
        for ctx in context_expansion:
            units = find_units_in_context(ctx, catalog)
            new_units.update(units)
        unit_expansion.update(new_units)

    return unit_expansion, context_expansion

Score: Energy  $E = -\log(|\text{unit\_expansion}| \times \log(|\text{context\_expansion}| + 1))$ 
Lower energy = larger expansion = better unit
```

3 Connection to Embeddings and Predictive Coding

3.1 Expansion as Implicit Embedding

Traditional embeddings: Map words to fixed vectors where similar words are nearby.

Our approach: The expansion *itself* is an embedding:

- Context defines a *substitution set* (the expansion)
- Elements in the set are interchangeable in these contexts
- This IS a grouping/clustering (dimensionality reduction)
- Members can predict each other (if one fits, others likely fit too)

3.2 Predictive Coding Interpretation

Large expansion $\implies$ high mutual predictability:

- If “go out” fits $\implies$ “hang out” probably fits

- If “hang out” fits $\implies$ “meet up” probably fits
- These units *predict each other*

This satisfies a predictive coding objective: maximize prediction accuracy within the expansion set.

3.3 Dynamic vs. Pre-Clustered

Pre-clustered approach:

```
# Offline: cluster contexts
catalog["go out"] = {
    "cluster_0": {contexts: [...], signature: vec_social},
    "cluster_1": {contexts: [...], signature: vec_motion}
}

# Parse time: match to clusters
if similarity(current_context, vec_social) > threshold:
    use cluster_0
```

Dynamic expansion approach:

```
# Parse time: expand both unit and context
expansion = bidirectional_expansion("go out", current_context)
# Clustering happens implicitly via expansion process
```

Advantage: Handles interpolation, novel contexts, soft boundaries.

4 Concrete Examples

4.1 Example 1: “go out” - Polysemy

4.1.1 Social Context

Sentence: Did you go out with Sarah

1. **Initial:** unit = “go out”, context = (“did you”, “with Sarah”)

2. **Context expansion:**

- “did you _ with Helen”
- “would you _ with me”
- “we _ together”
- “let’s _ on Friday”

3. **Unit expansion:**

- go out, hang out, meet up, get together, chat, connect

4. **Score:** $|\text{units}| \approx 15, |\text{contexts}| \approx 30 \implies E = -\log(15 \times \log(31)) \approx -4.1$

4.1.2 Motion Context

Sentence: I go out the door

1. **Initial:** unit = “go out”, context = (“I”, “the door”)
2. **Context expansion:**
 - “I _ the door”
 - “we _ the exit”
 - “please _ this way”
 - “you should _ now”
3. **Unit expansion:**
 - go out, walk out, exit, leave, step out, head out
4. **Score:** $|\text{units}| \approx 12, |\text{contexts}| \approx 25 \implies E = -\log(12 \times \log(26)) \approx -3.9$

Key point: Same surface form “go out”, different expansions, both recognized as valid units with context-appropriate meanings.

4.2 Example 2: “my money” vs. “lost my”

Sentence: I lost my money yesterday

4.2.1 Testing “my money” as unit

1. **Context expansion:**
 - “I lost _”
 - “I need _”
 - “where is _”
 - “I found _”
 - “I want _”
2. **Unit expansion:**
 - my money, your keys, his wallet, the car, my phone, your help, ...
3. **Score:** Large expansion (possessive NPs are common)

4.2.2 Testing “lost my” as unit

1. **Context expansion:**
 - “I _ money”
 - “I _ keys”
 - “you _ wallet” (but this changes to “your”)
2. **Unit expansion:**

- lost my, found my, need my, want my (limited, requires “my”)
3. **Score:** Small expansion (verb+determiner not freely substitutable)

Result: “my money” wins due to larger bidirectional expansion.

5 Implementation

5.1 Prototype Architecture

We implemented three versions:

5.1.1 Version 1: Simple Frequency-Based (`prototype_topdown_units.py`)

- Builds corpus catalog: 32k n-grams ($n=1..4$) with contexts
- Scores units by: $E = -\log(\text{freq}) - \text{compositionality_bonus} + \lambda \cdot \text{context_mismatch}$
- Top-down parsing: test span as unit before splitting
- **Success:** Finds “go out”, “my money”, “did you”, “the door”
- **Limitation:** Uses local context matching, not full expansion

5.1.2 Version 2: Explicit Bidirectional Expansion (`bidirectional_expansion_parser.py`)

- Stores: unit $\rightarrow$ contexts mapping AND context $\rightarrow$ units mapping
- Implements expansion algorithm as described
- **Challenge:** Memory explosion - storing all individual context instances infeasible at scale

5.1.3 Version 3: Aggregated Expansion (`bidir_simple.py`)

- Uses aggregated context distributions (Counter objects)
- Context similarity via Jaccard on top-k words
- Expansion limited by similarity threshold
- **Current status:** Working but needs refinement

5.2 Parsing Algorithm

```
def parse_topdown(span, tokens, catalog):
    # 1. TEST AS UNIT
    unit_expansion, context_expansion = \
        bidirectional_expansion(span.text, context, catalog)

    unit_energy = -log(len(unit_expansion) *
                       log(len(context_expansion) + 1))

    # 2. TEST ALL POSSIBLE SPLITS
    best_split_energy = infinity
```

```

for split_point in possible_splits(span):
    left = parse_topdown(left_span, tokens, catalog)
    right = parse_topdown(right_span, tokens, catalog)
    split_energy = left.energy + right.energy + penalty

    if split_energy < best_split_energy:
        best_split_energy = split_energy
        best_split = (left, right)

# 3. CHOOSE: unit vs. split
if unit_energy < best_split_energy:
    return UnitNode(span, unit_energy)
else:
    return SplitNode(span, best_split)

```

6 Current Challenges

6.1 Challenge 1: Scalability of Explicit Expansion

Problem: With 32k units in catalog, comparing all pairs for context similarity is $O(n^2)$.

For sentence parsing, we need:

- Fast lookup of similar contexts
- Fast retrieval of units fitting a context pattern

Current bottleneck: Iterating through all catalog entries for each candidate span.

6.1.1 Potential solutions:

1. **Pre-compute similarity graph:** Offline clustering of contexts
2. **Embedding-based retrieval:** Encode contexts as vectors, use approximate nearest neighbors
3. **Inverted index:** Index units by context features (e.g., top-k words)

6.2 Challenge 2: Context Similarity Metric

Current approach: Jaccard similarity on top-10 words from context distributions

Problems:

- Too loose: Many false positives (threshold = 0.15 gives 19k expansions)
- Too strict: Misses valid generalizations (threshold = 0.30 still gives 1.5k expansions)
- Ignores word order and syntactic structure

6.2.1 Example of false positive:

```
"go out" context: (left: [to, you, i], right: [with, and, the])
"a big" context: (left: [to, was, i], right: [and, the, for])
```

```
Jaccard similarity = 0.4 (overlaps on: to, i, and, the)
=> Incorrectly deemed similar!
```

6.3 Challenge 3: Representation of Context Patterns

Question: How should we represent a “context pattern”?

Current options tried:

1. **Exact tuples:** (w_{-3}, w_{-2}, w_{-1}) and (w_{+1}, w_{+2}, w_{+3})
 - Pro: Precise
 - Con: Doesn’t generalize to novel patterns
2. **Aggregated distributions:** Counter of all words seen in left/right contexts
 - Pro: Generalizes across instances
 - Con: Loses structural information
3. **Embeddings:** Encode context as vector
 - Pro: Captures semantic similarity
 - Con: Requires training/pre-trained model

6.4 Challenge 4: The Averaging Problem

Core issue: We want to *distinguish* different meanings (social vs. motion “go out”), but aggregation tends to *blend* them.

If we represent “go out” contexts as:

$$\begin{aligned}\text{context_distribution} &= \text{Counter}(\text{all left words}) \\ &= \{ \text{“to”} : 115, \text{“you”} : 81, \text{“I”} : 45, \dots \}\end{aligned}$$

This mixes social and motion contexts together. When we compute similarity, we can’t distinguish which meaning is active.

Desired: Context patterns should cluster by meaning

- Social cluster: “did you”, “would you”, “let’s”, “we”, ...
- Motion cluster: “I”, “please”, “you should”, + spatial objects

Question: Should clustering be:

1. **Pre-computed** (offline clustering of contexts into meaning groups)?
2. **Dynamic** (expand and naturally separate via similarity)?
3. **Implicit** (learned neural representations)?

7 Connection to Goertzel’s Hamiltonian Framework

The Causal-HNet paper (Goertzel, 2025) proposes a related approach:

7.1 Hamiltonian Energy for Relations

Define energy functions $E_r(p_i, p_j)$ for relation types r :

- $E_{\text{social}}(p_{\text{go}}, p_{\text{out}}) = -5p_i p_j + 1$
- $E_{\text{motion}}(p_{\text{go}}, p_{\text{out}}) = -4p_i p_j + 1$

Low energy when both nodes active and relation satisfied.

7.2 Dynamic Edge Prediction

From sentence embedding z , predict:

- Edge mask: $m_{ij}(z) \in [0, 1]$ (should this edge be active?)
- Relation weights: $w_{ij,r}(z)$ (what relation type?)

Key insight: Different sentence contexts $\implies$ different $z \implies$ different edge predictions.

7.3 Relation to Our Approach

Our expansion approach	Hamiltonian approach
Explicit expansion sets	Implicit via learned energies
Corpus statistics	Learned parameters
Maximize expansion size	Minimize energy
Interpret: substitutability	Interpret: relation satisfaction

Potential synthesis:

- Use expansion to *train* Hamiltonian energies
- Or: Use learned energies to *guide* expansion
- Or: Hybrid objective combining both

8 Next Steps and Open Questions

8.1 Immediate Technical Questions

1. **Scalability:** How to make bidirectional expansion efficient at scale?
 - Approximate methods (LSH, embeddings)?
 - Pre-computed indices?
 - Limit expansion depth/breadth?
2. **Similarity metric:** What’s the right way to measure context similarity?
 - Embeddings (BERT, word2vec)?

- Structural features (POS tags, dependency patterns)?
 - Learned metric?
3. **Representation:** How to represent contexts to preserve both generalization and discrimination?
- Hybrid: exact + aggregate?
 - Hierarchical: word-level + pattern-level?
 - Latent: learned representations?

8.2 Architectural Choices

Option A: Refine explicit expansion

- Better indexing/clustering
- Efficient retrieval structures
- Interpretable, corpus-grounded

Option B: Neural dynamic predictor

- Learn $z = \text{encode}(\text{sentence})$
- Predict expansions implicitly via network
- End-to-end trainable

Option C: Hybrid

- Use expansion concept for objective
- Use learned representations for efficiency
- Best of both worlds?

8.3 Theoretical Questions

1. **Convergence:** Does bidirectional expansion converge? Under what conditions?
2. **Uniqueness:** Can different segmentations yield similar expansion sizes?
3. **Compositionality:** How does expansion size relate to semantic compositionality?
4. **Learnability:** Can we learn to predict expansion size from sentence features?

9 Conclusion

We’ve developed a parsing approach based on a simple intuition: *syntactic units are those with large substitution sets across diverse contexts*.

The key innovation is **bidirectional expansion**: simultaneously expanding both the unit set (what can substitute) and context set (where they can substitute), enabling generalization to novel sentences.

Our prototype demonstrates the concept works:

- Successfully finds multi-word units: “go out”, “my money”, “did you”
- Prefers units with strong corpus support
- Handles polysemy through context-dependent expansion

The main challenge is **scalability**: efficient expansion at corpus scale requires better algorithms or learned representations.

This approach connects to:

- **Distributional semantics**: Meaning from substitution patterns
- **Predictive coding**: Large expansion = high mutual predictability
- **Hamiltonian frameworks**: Energy minimization over relational constraints
- **Freeman-style attention**: Substitutability as basis for grouping

We see this as a promising direction that grounds parsing in fundamental distributional principles rather than arbitrary local windows.